

Applicazione di un algoritmo Ant Colony Optimization al problema del Capacitated Vehicle Routing Problem

Marco Spampinato

Link alla repository: [GitHub - Progetto CVRP](#).

1 Introduzione

Il presente elaborato documenta il processo di sviluppo e analisi di un algoritmo della famiglia *Ant Colony Optimization* per la risoluzione del *Capacitated Vehicle Routing Problem* (CVRP), un classico problema di ottimizzazione NP-hard e generalizzazione del *Traveling Salesman Problem* (TSP).

L'algoritmo realizzato è basato su *Applying the ANT System to the Vehicle Routing Problem* [1], il quale propone un primo elegante e estensivamente descritto algoritmo *ACO* per la risoluzione del CVRP, estendendo l'algoritmo *Ant System* con l'applicazione di 2-Opt[2], al fine di eliminare archi sovrapposti nello stesso percorso, così da semplificare la soluzione e ridurre il costo.

È stata inoltre adottata l'euristica *MAX-MIN* [5], la quale limita il decadimento e la saturazione del feromone, infine è stata implementata la possibilità di eseguire l'algoritmo su più processori, come suggerito da [4], riducendo i tempi di esecuzione.

1.1 Capacitated Vehicle Routing Problem

Il *Capacitated Vehicle Routing Problem* (CVRP) è un problema di ottimizzazione combinatoria NP-hard che estende il classico problema del commesso viaggiatore (TSP), introducendo vincoli di capacità sulle risorse di trasporto.

Formalmente, il problema può essere modellato su un grafo completo $G = (V, E)$, dove:

- $V = \{v_0, v_1, \dots, v_n\}$ è l'insieme dei nodi, in cui il nodo v_0 rappresenta il deposito centrale e i restanti nodi $V \setminus \{v_0\} = \{v_1, \dots, v_n\}$ rappresentano i clienti.
- $E = \{(i, j) : i, j \in V\}$ è l'insieme degli archi o percorsi che collegano i nodi, a ciascuno dei quali è associato un costo di percorrenza d_{ij} (es. distanza o tempo).
- Ogni cliente $v_i \in V \setminus \{v_0\}$ richiede una quantità di merce pari a $q_i > 0$.
- Presso il deposito è disponibile una flotta di m veicoli identici, ciascuno avente una capacità massima di carico pari a Q .

L'obiettivo del CVRP è determinare un insieme di percorsi a costo totale minimo tali per cui:

1. Ogni cliente venga visitato esattamente una volta da un solo veicolo.
2. Ogni percorso inizi e termini obbligatoriamente presso il deposito centrale (v_0).
3. Per ciascun veicolo k , la somma delle richieste dei clienti serviti in quel viaggio non superi la capacità massima Q :

$$\sum_{i \in k} q_i \leq Q : \forall k \in \text{Veicoli} \quad (1)$$

In termini intuitivi, il problema consiste nel pianificare i percorsi di una flotta di furgoni per servire tutti i clienti sparsi sul territorio, minimizzando la strada totale percorsa ed evitando che un qualsiasi veicolo superi il proprio limite di carico massimo durante il tragitto.

1.2 Ant Colony Optimization

L'algoritmo realizzato appartiene alla famiglia *Ant Colony Optimization* (ACO); si definisce pertanto bio-ispirato in quanto simula il comportamento euristico delle formiche nell'approvvigionamento di risorse. Tale comportamento è fortemente influenzato dal **deposito di feromone** lungo i percorsi praticati verso il nido da parte delle formiche che trasportano il cibo. Questa sostanza agisce analogamente a una **memoria collettiva** e decentralizzata, influenzando le altre formiche della colonia a seguire i tracciati con maggiore concentrazione, fino a giungere alla fonte.

Il feromone è infine soggetto a *evaporazione*, ovvero una progressiva e naturale riduzione della sua concentrazione nel tempo. È stato osservato che le formiche scelgono generalmente i percorsi più brevi: ciò è dovuto al fatto che sui tracciati più lunghi il tempo di percorrenza maggiore permette una superiore evaporazione, mentre sui percorsi più corti il passaggio più frequente delle formiche tende a mantenere il feromone più *concentrato*, creando un **meccanismo di rinforzo positivo**.

2 Algoritmo realizzato

L'algoritmo realizzato genera ad ogni iterazione un numero K di soluzioni pari al numero di formiche della colonia, tali soluzioni sono influenzate dalle euristiche di visibilità e feromone. Generate le K soluzioni, una seconda fase, detta di *aggiornamento*, riduce la quantità di feromone globale lungo i percorsi tramite *evaporazione*, mentre rinforza i percorsi scelti dalle K soluzioni. L'algoritmo restituisce sempre la soluzione migliore trovata.

Generalmente, K è fissato al numero di clienti nel problema. La [Figura 1](#) mostra lo pseudocodice del ciclo principale dell'algoritmo.

Figura 1: Pseudocodice algoritmo ACO

```

1  while not terminated:
2      generate K solutions
3      store the best solution, if found
4      update pheromone
5  return best solution

```

2.1 Generazione delle soluzioni

Ogni formica virtuale genera una soluzione visitando tutti i nodi cliente, in particolare la formica è definita dal seguente stato:

- Una determinata posizione V_i ad ogni iterazione.
- Un carico Q_i , corrispondente al carico residuo al nodo V_i , dove vale sempre $Q_{depot} = Q_{start} = Q$, data Q la capacità o carico massimo definito dell'istanza di problema .

- Una memoria dell'insieme dei nodi da visitare Ω .
- Una memoria della lista dei percorsi, che chiameremo *mini-tour*.

Come esemplificato in [Figura 2](#), ogni formica avvia l'esplorazione partendo da un certo nodo iniziale V_{start} e un *mini-tour* vuoto; finché vi sono nodi non visitati (ad eccezione del nodo di partenza e il deposito), essa seleziona il prossimo nodo V_j tra i nodi non visitati, secondo una certa probabilità P_{ij} . Qualora la domanda del nodo V_j superasse il carico Q_i della formica, la formica torna al deposito, resetta il suo carico a Q e inizia un nuovo *mini-tour*; altrimenti visita il nodo V_j , diminuendo il suo carico della domanda del nodo visitato, e aggiungendo V_j al *mini-tour*.

Figura 2: Pseudocodice generazione di una soluzione

```

1  unvisited_nodes = all_nodes \ { Vdepot, Vstart }
2  if Vstart != Vdepot: visit Vstart
3  while |unvisited_nodes| > 0:
4      select unvisited node Vj according to probability Pij
5      if Vj's demand > load:
6          visit depot and restore load
7      else:
8          unvisited_nodes = unvisited_nodes \ {Vj}
9          visit node Vj and unload Vj's demand
10 visit depot
11 return solution

```

Vi sono tre strategie per scegliere V_{start} :

- Usare come punto di partenza il deposito: $V_{start} = V_{depot}$.
- Assegnare il punto di partenza di ogni formica a un cliente $V_{start} = V_k$.
- Usare un nodo random $V_{start} = V_{rand()}$.

In letteratura, è stato sperimentalmente documentato che assegnare a ogni cliente una formica, determina un risultato di maggiore qualità.

2.1.1 Scelta dei nodi da visitare

La scelta del prossimo nodo da visitare è data dalla seguente distribuzione di probabilità:

$$P_{ij} = \begin{cases} \frac{[\tau_{ij}]^\alpha [\eta_{ij}]^\beta}{\sum_{h \in \Omega} [\tau_{ih}]^\alpha [\eta_{ih}]^\beta} & \text{se } j \in \Omega \\ 0 & \text{se } j \notin \Omega \end{cases}$$

Dove:

- τ_{ij} è la quantità di feromone presente nell'arco (i, j) .
- η_{ij} è la costante di visibilità, inversamente proporzionale alla distanza tra V_i e V_j , i.e. $\eta_{ij} = d_{ij}^{-1}$.
- α e β sono costanti che influenzano rispettivamente la scelta tra feromone e visibilità, ovvero pesano la scelta tra $P_{ij} \sim \tau_{ij}$ e $P_{ij} \sim \eta_{ij}$.
- Ω è l'insieme dei nodi non ancora visitati.

Al fine di migliorare la qualità dell'algoritmo, sono state adottate altre due euristiche usate in [1], le misure di *risparmio* e *capacità*, pertanto la effettiva probabilità realizzata è:

$$P_{ij} = \begin{cases} \frac{[\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta \cdot [\mu_{ij}]^\gamma \cdot [\kappa_{ij}]^\lambda}{\sum_{h \in \Omega} [\tau_{ih}]^\alpha \cdot [\eta_{ih}]^\beta \cdot [\mu_{ih}]^\gamma \cdot [\kappa_{ih}]^\lambda} & \text{se } j \in \Omega \\ 0 & \text{se } j \notin \Omega \end{cases}$$

Le due nuove euristiche sono:

- La misura di *risparmio* $\mu_{ij} = d_{i0} + d_{0j} - d_{ij}$ quantifica quanto è conveniente visitare un nodo V_j dopo aver visitato V_i , basandosi sulla distanza, è influenzata dall'esponente γ .
- La misura di *capacità* $\kappa_{ij} = 1 - \frac{Q_i - q_j}{Q}$ quantifica quanto è conveniente visitare un nodo V_j basandosi su quanto carico Q_i ho ancora disponibile. Si precisa che la formulazione differisce da [1], in quanto il nostro algoritmo usa come metrica il carico residuo, anziché la capacità residua. Inoltre per $\kappa_{ij} > 1$, la soluzione è *unfeasable*, ovvero non è possibile soddisfare la richiesta.

2.1.2 Soluzione finale

Una volta visitati tutti i nodi, la formica virtuale costruisce una soluzione comprendente di:

- Costo totale della soluzione, intesa come lunghezza complessiva di tutti i mini-tour.
- Un insieme di tutti i mini-tour appartenenti alla soluzione.

Al fine di migliorare il risultato, come proposto in [1], è possibile eseguire sui mini-tour l'algoritmo *2-Opt*, una euristica di ricerca locale che migliora un percorso eliminando due collegamenti e invertendo la tratta racchiusa, al fine di eliminare gli incroci e quindi ridurre la lunghezza.

2.2 Aggiornamento del feromone

Il feromone, una volta generate le K soluzioni, viene aggiornato dalla seguente formula:

$$\tau_{ij} \leftarrow (1 - \rho) \cdot \tau_{ij} + \Delta\tau_{ij}^{(S)} + \sigma\Delta\tau_{ij}^*$$

Dove:

- ρ è il coefficiente di evaporazione, regola quanto feromone viene perso in ogni iterazione
- $\Delta\tau_{ij}^{(S)}$ è il deposito di feromone dipendente dalle soluzioni generate nella stessa generazione, vi sono state realizzate due strategie di aggiornamento:
 - Deposito usando solo i percorsi usati dalla soluzione migliore della iterazione, questa strategia è generalmente la migliore.
 - Deposito usando i percorsi realizzati da tutte le soluzioni.
- $\sigma\Delta\tau_{ij}^*$ è invece un contributo della migliore soluzione trovata, in letteratura prende il nome di *formiche elitiste*, e simula il deposito di σ formiche usando la soluzione migliore globale. Solitamente $\sigma = K$.

2.2.1 MAX-MIN Ant System

Da [5], MAX-MIN è una euristica atta a limitare il decadimento e l'accumulo del feromone, restringendo dunque i valori della matrice dei feromoni all'interno di un determinato range, rendendo più stabili i calcoli e favorendo una maggiore esplorazione dello spazio delle soluzioni. L'articolo originale propone un metodo per definire τ_{max} e τ_{min} , tuttavia la formulazione di quest'ultimo è alquanto ostica, pertanto una sua semplificazione viene adottata.

$$\tau_{max} = \frac{1}{\rho \cdot C^*}, \quad \tau_{min} = \frac{\tau_{max}}{2 \cdot K}$$

Dove C^* è la soluzione migliore globale. MAX-MIN viene applicato successivamente l'aggiornamento dei feromoni.

Una opzione non esplorata nell'analisi dei risultati, è lo *smussamento* o *smoothing*, una operazione di normalizzazione, che tende ad appiattire i livelli di feromoni secondo un parametro $\delta \in [0, 1]$:

$$\tau_{ij} \leftarrow \tau_{ij} + \delta \cdot (\tau_{max} - \tau_{ij})$$

3 Implementazione

L'algoritmo è stato implementato nel linguaggio di programmazione Python, facendo uso della libreria *numpy* per varie utilità matematiche. Le classi e script prodotti sono:

- La classe **Instance** cui responsabilità sono il caricamento dei file di istanza del problema in formato `.vrp`.
- La classe **Solution** che descrive una soluzione del problema, con funzionalità di verifica e serializzazione del soluzione.
- La classe astratta **BaseSolver**, la quale definisce una interfaccia comune ai vari algoritmi di risoluzione implementabili.
- La classe **ACOSolver**, la quale eredita **BaseSolver**, implementa l'algoritmo ACO come descritto precedentemente.
- La classe **ACO_MPSolver**, estensione di **ACOSolver**, in grado di eseguire su più core l'algoritmo.
- Lo script `grinder.py`, il quale contiene il programma di automazione per eseguire l'algoritmo su diverse istanze e diversi profili di parametri.
- Lo script `utils.py` che contiene diverse utilità, tra cui la possibilità di stampare a video l'istanza di un problema e una sua soluzione.
- Gli script `main.py` e `grind.py`, i quali forniscono un accesso da riga di comando all'implementazione.

Per maggiori informazioni, si invita a consultare il *readme* dei sorgenti forniti.

3.1 Stabilità numerica

Uno dei problemi implementativi affrontati è stato quello della stabilità numerica: calcolare la probabilità P_{ij} al fine di selezionare un nodo da visitare, ha portato a diversi errori di runtime, quali *underflow* e *overflow*, dovuti ai limiti di precisione in virgola mobile.

Una operazione comune nel machine learning è la normalizzazione di probabilità in scala logaritmica, come il softmax:

$$p_i = \frac{\exp(x_i)}{\sum_{n=1}^N \exp(x_n)} \quad t.c. \sum_{i=1}^N p(x_i) = 1$$

Tale operazione nell'atto pratico è risolta calcolando il logaritmo di tale probabilità:

$$p_i = \exp(x_i - \log(\sum_{n=1}^N \exp(x_n))) = \exp(x_i - LSE(x_1 \dots x_n))$$

Ciò garantisce che $\max(p_i) \leq \exp(0) = 1$ e $\sum_{n=1}^N p_i = 1$ senza problemi di stabilità numerica. Questa strategia matematica prende il nome di **Log-Sum-Exp trick**[3], ed è stata adottata per risolvere l'analogo calcolo di P_{ij} .

In particolare possiamo scrivere:

$$P_{ij} = \exp \begin{cases} \log \frac{[\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{l \in \Omega} [\tau_{il}]^\alpha \cdot [\eta_{il}]^\beta} & \text{se } j \in \Omega \\ -\infty & \text{se } j \notin \Omega \end{cases}$$

E ricondurci a quanto visto prima:

$$x_{ij} = \log([\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta) = \alpha \cdot \log \tau_{ij} + \beta \cdot \log \eta_{ij}$$

$$P_{ij} = \exp \begin{cases} \log \frac{\exp(x_{ij})}{\sum_{h \in \Omega} \exp(x_{ih})} & \text{se } j \in \Omega \\ -\infty & \text{se } j \notin \Omega \end{cases}$$

$$P_{ij} = \begin{cases} \exp(x_{ij} - LSE(x_{i1} \dots x_{ih})) & \text{se } j \in \Omega \\ 0 & \text{se } j \notin \Omega \end{cases}$$

3.2 Computazione parallela

La classe *ACO_MPSolver* implementa una specializzazione dell'algoritmo base in grado di eseguire il calcolo su più core, usando la libreria *multiprocessing* e la sua classe *Pool*, quale mette a disposizione un sistema per l'invocazione da parte dell'utente di funzioni Python su processi fork.

La classe sovrascrive il metodo della classe padre *ACOSolver* responsabile della simulazione e generazione risultati, affinché il carico possa essere distribuito in blocchi su più processori usando la classe *Pool*, riducendo significativamente i tempi di iterazione.

4 Sperimentazione e risultati

L'algoritmo è stato testato su diverse istanze di problema assegnate dal docente. Ogni istanza è stata valutata su 7 differenti profili di parametri dell'algoritmo, e ogni profilo è stato valutato 5 volte sulla istanza data. Il criterio di terminazione assegnato è il numero di valutazioni della funzione fitness $FE = 3.5 \cdot 10^5$, poiché l'algoritmo computa $K = \text{numeroclienti}$ soluzioni ogni iterazione, il numero di iterazioni massime per istanza allora è pari a $N = \lceil \frac{FE}{K} \rceil = \lceil \frac{3.5 \cdot 10^5}{K} \rceil$.

I profili usati derivano da [1]. In particolare il profilo *NN* corrisponde a una ricerca completamente stocastica, come nearest neighbour, infatti non vi è influenza del feromone. I profili della classe *AS* e *HAS* invece sono gli algoritmi ACO o *Ant System*, dal nome originale dei primi algoritmi, e sono influenzati del feromone, gli algoritmi *HAS* (*Hybrid Ant System*) si contraddistinguono per eseguire *2-Opt* su ogni mini-tour. Per tutti i profili, ad eccezione di *NN*, è

Profilo	α	β	γ	λ
NN	0	1	0	0
AS	1	5	0	0
HAS	1	5	0	0
HAS-SAV	1	5	5	0
HAS-CAP	1	5	0	5
HAS-1	1	5	5	5
HAS-5	5	5	5	5

Tabella 1: Profili utilizzati

Instanza	Profilo	KBS	Best	Err %	Media	Std	Avg. It.
A-n45-k7	HAS	1146.0	1175.6	2.583%	1181.86	5.62	5843.2
A-n60-k9	HAS-5	1354.0	1372.94	1.399%	1394.51	17.97	3580.6
A-n80-k10	AS	1763.0	1824.06	3.463%	1839.3	15.81	2704.6
B-n56-k7	HAS-5	707.0	755.62	6.877%	764.78	8.8	3663.6
B-n66-k9	AS	1316.0	1338.72	1.726%	1341.88	2.38	2954.0
B-n78-k10	HAS-5	1221.0	1273.12	4.269%	1283.98	9.99	2495.0
E-n101-k14	HAS-SAV	1071.0	1141.05	6.540%	1151.89	8.94	2621.6
E-n76-k8	HAS-1	735.0	758.97	3.261%	771.25	10.78	3369.8
P-n101-k4	HAS-1	681.0	697.04	2.356%	705.09	4.84	2021.8
P-n50-k10	HAS-5	696.0	707.03	1.585%	720.84	10.26	1506.2

Tabella 2: Risultati migliori. In grassetto i risultati più importanti.

stato usato un numero $\sigma = K$ di formiche elitarie e costante di evaporazione pari a $\rho = 0.25$, il deposito di feromone è stato consentito solo alle soluzioni migliori della iterazione, e infine è stato usato il *MAX-MIN* per limitare la matrice dei feromoni τ .

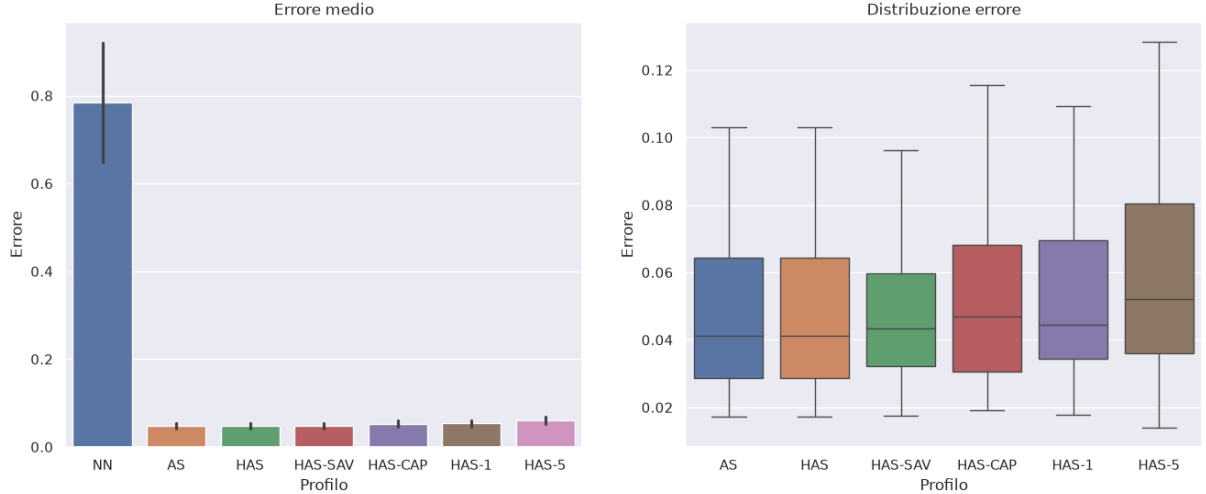


Figura 3: Errore medio e distribuzione per profilo.

La [Tabella 2](#) mostra i risultati migliori per ogni istanza del problema e il profilo che ha dato il risultato migliore. La colonna **KBS** indica la *Known Best Solution*, la soluzione migliore conosciuta in letteratura; la colonna **Best** indica la soluzione migliore trovata dall'algoritmo; **Err %** indica l'errore percentuale della soluzione trovata rispetto la soluzione ottima KBS; **Media** e **Std** indicano rispettivamente la media e deviazione standard delle soluzioni ottenute

da quel profilo sulla istanza; infine **Avg. it.** indica il numero di iterazioni medie del profilo per arrivare alla migliore soluzione.

Analogamente, i grafici in figura [Figura 3](#) mostrano rispettivamente l'errore medio ottenuto e la distribuzione dell'errore per ogni profilo di parametro usando un *boxplot*.

Come si può notare dal **primo grafico**, il profilo *NN* presenta un errore medio molto alto, mentre gli errori medi dei profili *ACO/AS* sono abbastanza comparabili.

Nel **secondo grafico** in [Figura 3](#) sono messe a confronto le distribuzioni di errore dei vari profili, ad eccezione di *NN*: i profili *AS* e *HAS* hanno una identica distribuzione; pertanto, l'uso di *2-Opt*, almeno per i dati parametri $\alpha = 1, \beta = 5$, non sembra avere alcun effetto sulle date istanze; il profilo *HAS-SAV*, che fa uso dell'euristica del risparmio, presenta la distribuzione con il range più stabile, al contrario di *HAS-CAP*, i cui risultati sono lievemente più inconsistenti rispetto ai profili precedenti, a indicare che l'euristica della capacità non sembra particolarmente contribuire a risultati migliori; il profilo *HAS-1*, invece, unisce entrambe le euristiche di risparmio e capacità, migliorando lievemente su *HAS-CAP* e ottenendo comunque i risultati migliori su due istanze di problema (*E-n76-k8*, *P-n101-k4*); infine il profilo *HAS-5*, il quale ha la più alta influenza di feromone tra tutti i profili, presenta la distribuzione di errore più grande, ottiene però i migliori risultati in ben quattro istanze del problema, in due di queste ottiene l'errore più basso in assoluto.

Si precisa che con tutti i profili, ad eccezione di *NN*, è stato ottenuto il numero minimo di mini-tour in tutte le run; in nessuna delle istanze è stata calcolata la soluzione *KBS*.

4.1 Analisi su tre istanze del problema

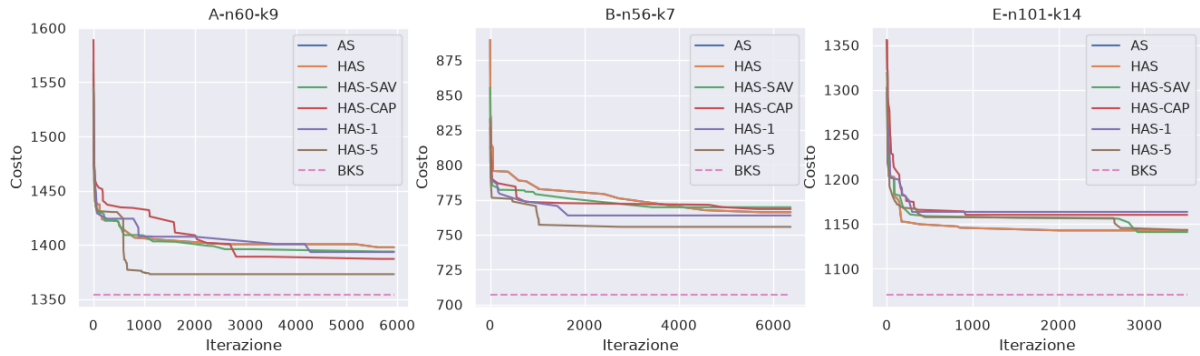


Figura 4: Curve di convergenza delle istanze A-n60-k9, B-n56-k7, E-n101-k14.

È stato scelto di approfondire l'analisi delle istanze *A-n60-k9*, *B-n56-k7*, *E-n101-k14*. Le curve di convergenza sono mostrate in [Figura 4](#), per ogni istanza del problema sono stati messi a confronto i vari profili, ad eccezione di *NN*, la linea tratteggiata color porpora rappresenta la *Known Best Solution* dell'istanza.

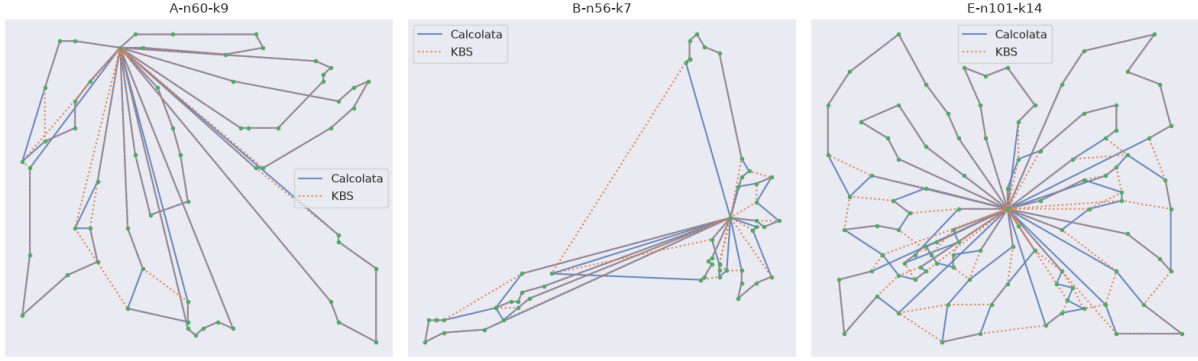


Figura 5: Soluzioni migliori delle istanze A-n60-k9, B-n56-k7, E-n101-k14.

L'algoritmo generalmente produce buoni risultati con un basso numero di iterazioni: il profilo *HAS-5* converge molto rapidamente nelle prime due istanze, dove le successive iterazioni generalmente non riconducono a risultati migliori; mentre nella terza, *HAS* e *AS* hanno una convergenza più rapida, con *HAS-5* e *HAS-SAV* che migliorano significativamente nelle ultime 1000 iterazioni, e quest'ultimo profilo produce un risultato marginalmente migliore. In Figura 5 sono confrontati i percorsi ottenuti con i percorsi della soluzione migliore, è immediato notare come nelle prime istanze i nodi sono generalmente clusterizzati (molto evidente nella seconda), mentre nella terza i nodi sono distribuiti più uniformemente e il deposito è localizzato in posizione centrale.

5 Conclusioni

Il presente progetto ha permesso la realizzazione di un algoritmo *Ant Colony Optimization* per risolvere il problema del *Capacited Vehicle Routing Problem*, basandosi su proposte e risultati della letteratura scientifica esistente. Dove possibile, è stato scelto di combinare diverse strategie: alcune intrinseche all'algoritmo, come le misure di risparmio e capacità, la *MAX-MIN Ant System*; altre implementative, come il *Log-Sum-Exp trick* e il calcolo multi processore con la libreria *multiprocessing*.

La metodologia di lavoro ha privilegiato la realizzazione dell'algoritmo di risoluzione per il problema assegnato, a discapito della ricerca esaustiva dei parametri, ottenendo comunque buoni risultati con i profili suggeriti dalla letteratura [1].

5.1 Sviluppi Futuri

Nonostante i buoni risultati ottenuti e la solidità dell'implementazione prodotta, si suggeriscono eventuali evoluzioni:

- Una più **estesa esplorazione dei parametri** in particolare il fine-tuning del parametro di evaporazione ρ e l'influenza del feromone α , studiare l'influenza dello *smoothing* del feromone [5] nei risultati, parametro non utilizzato nella sperimentazione. Inoltre l'**introduzione di nuove euristiche** per la ricerca locale alternativi al *2-Opt*, come il *3-Opt* o l'algoritmo *Kernighan-Lin*, potrebbe significativamente migliorare la qualità delle soluzioni.
- **Migliorare i tempi di esecuzione** è di vitale importanza per supportare l'attività di ricerca dei parametri e il calcolo di soluzioni, pertanto si suggerisce di esplorare un **approccio vettorizzato** all'algoritmo, che sfrutti quanto il più possibile le istruzioni SIMD dei processori, offerti già trasparentemente da *numpy* per molte operazioni sui tipi della libreria: una tale evoluzione, inoltre, renderebbe triviale adattare il codice per essere eseguito su unità di calcolo massivamente parallele, come le GPU.

Riferimenti bibliografici

- [1] Bernd Bullnheimer, Richard Hartl e Christine Strauss. “Applying the ANT System to the Vehicle Routing Problem”. In: lug. 1997, pp. 285–296. ISBN: 978-0-7923-8369-7. DOI: [10.1007/978-1-4615-5775-3_20](https://doi.org/10.1007/978-1-4615-5775-3_20).
- [2] G. A. Croes. “A Method for Solving Traveling-Salesman Problems”. In: *Operations Research* 6.6 (1958), pp. 791–812. DOI: [10.1287/opre.6.6.791](https://doi.org/10.1287/opre.6.6.791). eprint: <https://doi.org/10.1287/opre.6.6.791>. URL: <https://doi.org/10.1287/opre.6.6.791>.
- [3] Gregory Gundersen. *The Log-Sum-Exp Trick*. Gregory Gundersen Blog. 9 Feb. 2020. URL: <https://gregorygundersen.com/blog/2020/02/09/log-sum-exp/> (visitato il giorno 15/07/2026).
- [4] Petr Stodola et al. “Using the Ant Colony Optimization algorithm for the Capacitated Vehicle Routing Problem”. In: *Proceedings of the 16th International Conference on Mechatronics - Mechatronika 2014*. Dic. 2014, pp. 503–510. DOI: [10.1109/MECHATRONIKA.2014.7018311](https://doi.org/10.1109/MECHATRONIKA.2014.7018311).
- [5] Thomas Stützle e Holger H. Hoos. “MAX–MIN Ant System”. In: *Future Generation Computer Systems* 16.8 (2000), pp. 889–914. ISSN: 0167-739X. DOI: [https://doi.org/10.1016/S0167-739X\(00\)00043-1](https://doi.org/10.1016/S0167-739X(00)00043-1). URL: <https://www.sciencedirect.com/science/article/pii/S0167739X00000431>.